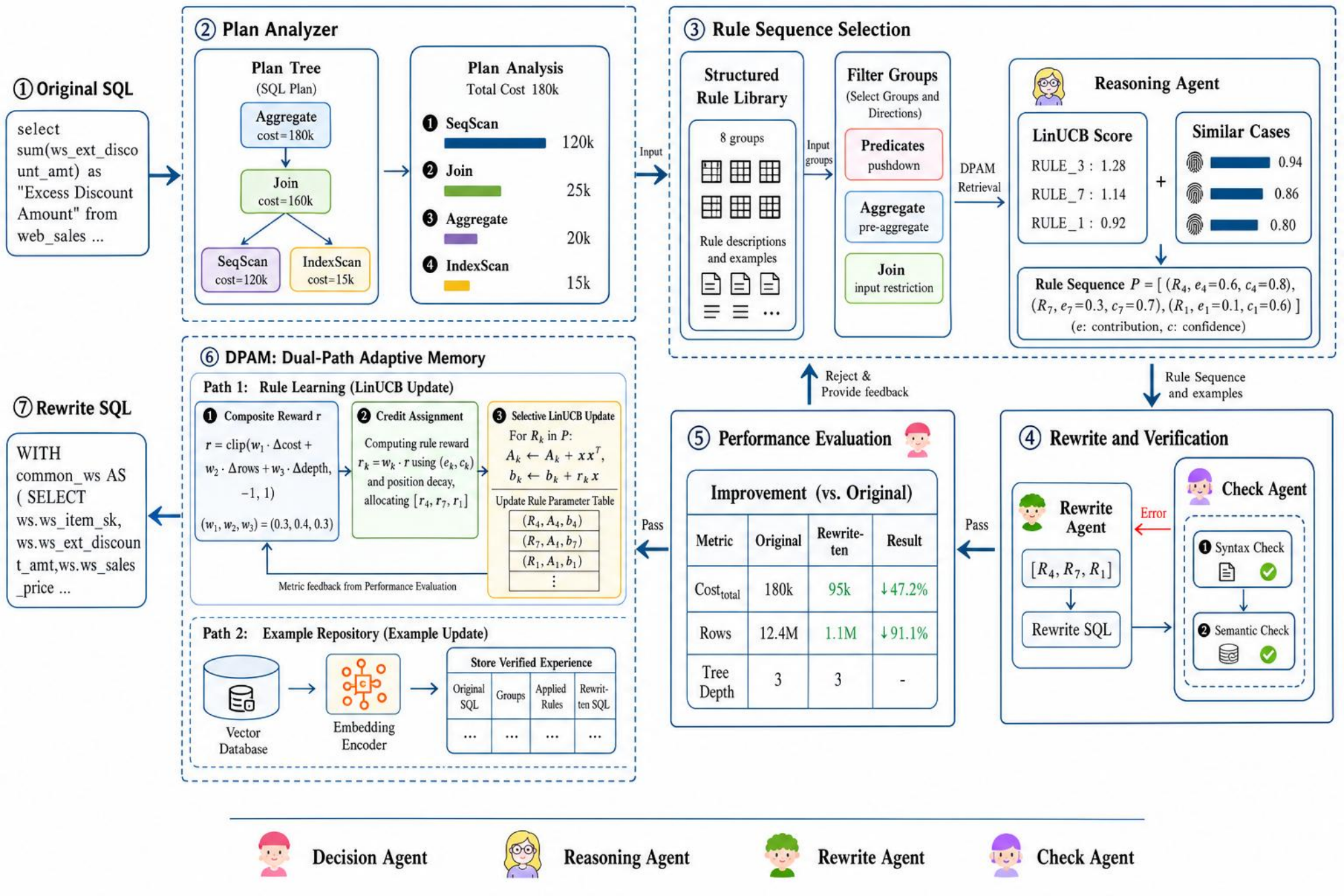

① Original SQL
select sum(ws_ext_discount_amt) as "Excess Discount Amount" from web_sales ...
② Plan Analyzer
Plan Tree
(SQL Plan)
Aggregate cost=180k
Join cost=160k
SeqScan cost=120k
IndexScan cost=15k
Plan Analysis
Total Cost 180k
1 SeqScan 120k
2 Join 25k
3 Aggregate 20k
4 IndexScan 15k
Input
③ Rule Sequence Selection
Structured Rule Library
8 groups
Rule descriptions and examples
Input groups
Filter Groups
(Select Groups and Directions)
Predicates pushdown
Aggregate pre-aggregate
Join input restriction
DPAM Retrieval
Reasoning Agent
LinUCB Score
RULE_3 : 1.28
RULE_7 : 1.14
RULE_1 : 0.92
+
Similar Cases
0.94
0.86
0.80
Rule Sequence P = [(R_4, e_4=0.6, c_4=0.8), (R_7, e_7=0.3, c_7=0.7), (R_1, e_1=0.1, c_1=0.6)]
(e: contribution, c: confidence)
Reject & Provide feedback
Rule Sequence and examples
⑥ DPAM: Dual-Path Adaptive Memory
Path 1: Rule Learning (LinUCB Update)
1 Composite Reward r
$r = \text{clip}(w_1 \cdot \Delta\text{cost} + w_2 \cdot \Delta\text{rows} + w_3 \cdot \Delta\text{depth}, -1, 1)$
$(w_1, w_2, w_3) = (0.3, 0.4, 0.3)$
2 Credit Assignment
Computing rule reward $r_k = w_k \cdot r$ using (e_k, c_k) and position decay, allocating $[r_4, r_7, r_1]$
3 Selective LinUCB Update
For R_k in P:
$A_k \leftarrow A_k + xx^T$,
$b_k \leftarrow b_k + r_k x$
Update Rule Parameter Table
(R_4, A_4, b_4)
(R_7, A_7, b_7)
(R_1, A_1, b_1)
Metric feedback from Performance Evaluation
Path 2: Example Repository (Example Update)
Vector Database
Embedding Encoder
Store Verified Experience
Original SQL | Groups | Applied Rules | Rewritten SQL
⑤ Performance Evaluation
Improvement (vs. Original)
Metric | Original | Rewritten | Result
$\text{Cost}_{\text{total}}$ | 180k | 95k | ↓47.2%
Rows | 12.4M | 1.1M | ↓91.1%
Tree Depth | 3 | 3 | -
Pass
④ Rewrite and Verification
Rewrite Agent
$[R_4, R_7, R_1]$
Rewrite SQL
Error
Check Agent
1 Syntax Check
2 Semantic Check
Pass
⑦ Rewrite SQL
WITH common_ws AS (SELECT ws.ws_item_sk, ws.ws_ext_discount_amt,ws.ws_sales_price ...
Decision Agent
Reasoning Agent
Rewrite Agent
Check Agent